Batch: _____	Roll No.: _____
Experiment / assignment / tutorial No. Mini Project (Phase 1)	

Title: Mini Project - Phase 1: MVP Definition and Full-Stack Foundation.

AIM: To convert the already-selected institute AI case study into a manageable full-stack MVP and establish the first working React -> Spring Boot -> Database workflow.

Objectives:

- To define a focused problem statement, users and core features for the selected case study.
- To design the full-stack architecture, entities and REST API requirements.
- To establish the project repository and frontend/backend structure.
- To implement and demonstrate one end-to-end feature with persistent data storage.
- To specify the planned AI/intelligent feature and its integration point for a later phase.

**(Students have to perform the assigned task within the project team and demonstrate the same).*

Resources used:

Development Workstations with Windows 11 x64, Java 21 LTS JDK, Node.js runtime, Maven build system, Supabase Cloud PostgreSQL managed database instance, and Git/GitHub source code management.

Tools and Technologies Used

- **- Frontend:** React, JavaScript (ES6+), HTML5, Tailwind CSS, Lucide React icons, and Recharts.
- **- Backend:** Java 21 LTS, Spring Boot 3.3.4, Spring Web (REST controllers), Spring Data JPA (Hibernate ORM), Spring Validation, and Spring Boot Actuator.
- **- Database:** PostgreSQL (Cloud PostgreSQL 15+ hosted on Supabase with SSL and HikariCP connection pooling).
- **- Version Control:** Git and GitHub (https://github.com/Shravani-3005/Java-Ev-battery-project).
- **- Development Tools:** Visual Studio Code, IntelliJ IDEA Ultimate, Google Chrome DevTools, Postman, and OpenAPI 3.0 / Swagger UI.
- **- Project Context:** KJS-CES-02 – AI-Enabled Micro-channel Battery Thermal Management System for Electric Vehicles.

Expected OUTCOME of Phase 1:

CO3: Integrate front-end and back-end components using APIs and state management techniques.

CO4: Design, develop, and query relational and NoSQL databases for full-stack applications.

Students will be able to define a manageable MVP for the selected case study, design its full-stack architecture and data/API model, and demonstrate one working React -> Spring Boot -> Database workflow committed to the project repository.

Outputs to be attached / recorded:

1. Project title and selected institute case study/use-case ID.
2. One-sentence MVP statement, user roles and 3-5 core features.
3. Architecture diagram showing React, Spring Boot, database and planned AI/intelligent component.
4. Entity/resource design and REST endpoint table.
5. GitHub repository link and evidence of meaningful commits.
6. Screenshot of the working React interface.
7. Evidence of a successful React -> Spring Boot -> Database operation and persistent record.
8. Brief specification of the planned AI/intelligent feature and its integration point.

Books/ Journals/ Websites referred:

1. Course material and documentation for React, Spring Boot, database integration and REST APIs.
2. Institute AI use-case material corresponding to the case study selected by the team (KJS-CES-02).
3. Craig Walls, 'Spring Boot in Action', Manning Publications, 2016; Spring Data JPA Reference Guide.
4. Official React Documentation (https://react.dev) and Vite Build Guide (https://vitejs.dev).
5. Bandhauer, T. M., Garimella, S., & Fuller, T. F., 'A Critical Review of Thermal Issues in Lithium-Ion Batteries', Journal of The Electrochemical Society, 158(3), R1-R25.

Pre Lab/ Prior Concepts:

React components/hooks, Spring Boot REST controllers, HTTP methods and JSON, database CRUD, Git/GitHub, JWT security and Swagger/OpenAPI from the completed laboratory experiments.

Project flow: User -> React Frontend -> Spring Boot REST API -> Database, with an AI/intelligent component integrated at a suitable point in a later phase.

- **1. Component-Based Frontend Architecture:** React 18 decomposes the UI into functional components with useState for reactive state and useEffect for async telemetry. Axios manages HTTP calls with unified interceptors.
- **2. Spring Boot RESTful Microservice Design:** DispatcherServlet maps requests to @RestController endpoints. Services annotate business logic with @Service, decoupling computation from HTTP serialization.
- **3. Relational Database Modeling & Persistence:** PostgreSQL provides relational integrity, atomic transactions (ACID), and foreign key validation. Tables model packs, readings, and cells, indexed for time-series querying.
- **4. EV Battery Thermal Management Principles:** Lithium-ion cells (LFP, NMC) operate safely within 15°C–35°C. Elevated temperatures (>45°C) degrade capacity, while temperatures above 55°C risk thermal runaway. Micro-channel cold plates transfer heat via convective liquid coolant. Monitoring coolant differentials (Tout - Tin), flow rate, and cell temperature variance (ΔT) enables early anomaly detection.

Implementation Details:

Phase 1 System Design

Architecture: React frontend -> JSON/HTTP -> Spring Boot REST API -> persistent database; planned AI/intelligent component -> result/advisory/classification -> application workflow.

Students have to write stepwise details of implementation for the following tasks:

Task 1: Project Metadata, MVP Definition, Users, and Core Features

- **Project Title:** THERMOCORE – EV Battery Thermal Intelligence & Monitoring System
- **Selected Institute Case Study / Use-Case ID:** KJS-CES-02 – AI-Enabled Micro-channel Battery Thermal Management System for Electric Vehicles
- **One-Sentence MVP Statement:** "Our system helps EV battery systems engineers and fleet telematics operators to monitor real-time battery pack thermal states and coolant dynamics by ingesting multi-cell sensor telemetry into a Spring Boot processing pipeline, evaluating thermal runaway risk via automated rule-driven heuristics, and visualizing live hotspot distribution and cooling recommendations in an industrial React dashboard."

Primary Users / Roles:

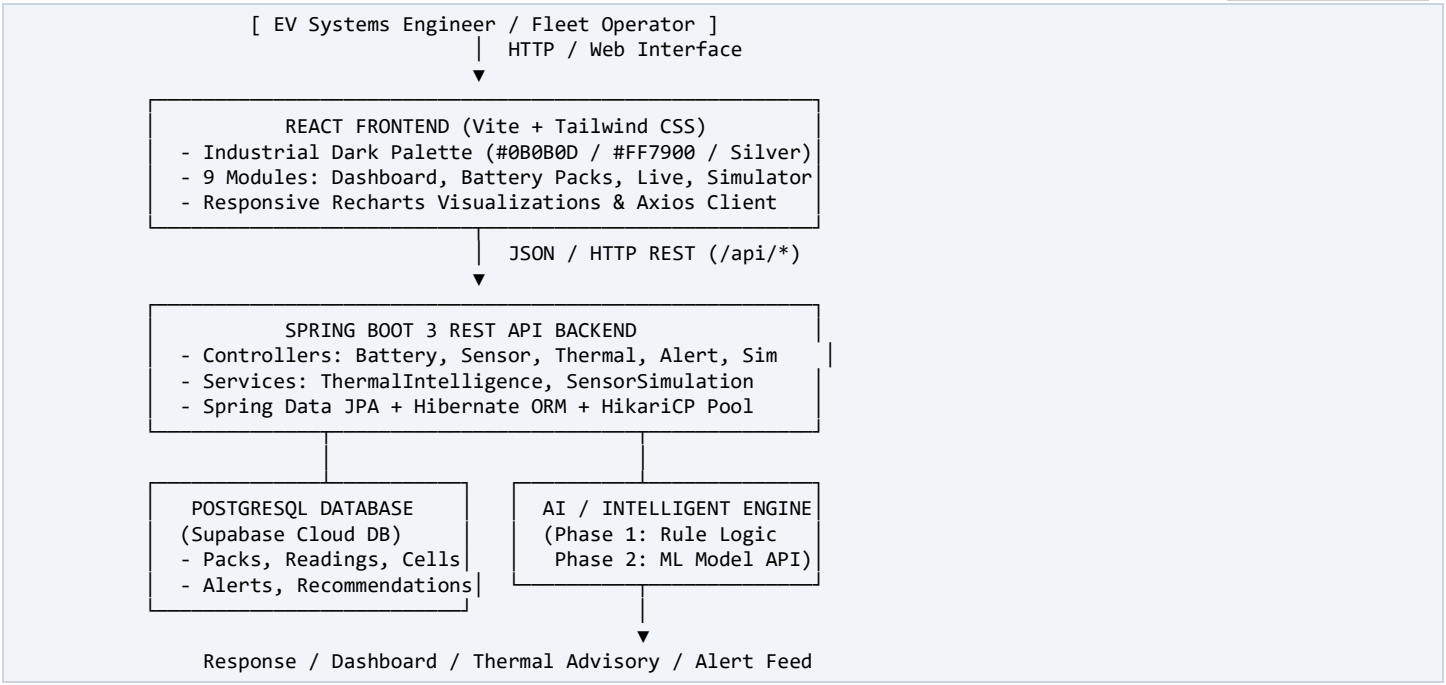
- **1. EV Battery Thermal Systems Engineer:** Analyzes pack thermal gradients, evaluates cooling channel heat dissipation, tunes safety threshold configurations, and inspects cell-level hotspot warnings.
- **2. Fleet Operations & Telematics Technician:** Monitors live vehicle fleet health, receives critical overheat notifications, acknowledges operational alerts, and dispatches preventive maintenance requests.
- **3. Battery Testing & Validation Engineer:** Configures simulated operating cycles (aggressive discharge, rapid DC fast charging, coolant pump failure) to study pack thermal dynamics.

Three to Five Core Features:

- **1. Real-Time Battery Telemetry Ingestion & Monitoring:** Continuous monitoring of pack voltage, current, average temperature, coolant inlet/outlet temperatures, flow rate (LPM), State of Charge (SOC), and State of Health (SOH).
- **2. Granular Cell Hotspot Detection & Heatmap Visualization:** Real-time monitoring of 96 to 144 individual cell temperatures per pack, calculating maximum temperature differences ($\Delta T = T_{max} - T_{min}$) and detecting localized overheating.
- **3. Automated Thermal Risk Evaluation & Alert Management:** Algorithmic classification into NORMAL, MODERATE, HIGH RISK, and CRITICAL levels with automated alert logging and technician acknowledgment workflows.
- **4. Active Micro-Channel Cooling Action Recommendations:** Rule-based cooling advisory recommending target coolant flow rate adjustments (LPM), radiator fan speed percentages, and powertrain power derating limits.
- **5. Multi-Scenario What-If Thermal Simulator:** Interactive simulation engine allowing operators to test normal driving, high-speed discharge, rapid DC charging, overheating stress, and coolant pump failures.
- **End-to-End Implementation Target for Today:** Battery Pack Management & Telemetry Ingestion (adding, viewing, updating packs and persisting time-series sensor observations in PostgreSQL).

Task 2: Proposed Full-Stack Architecture Diagram

The architecture strictly adheres to the institute specification while integrating the verified components:



Task 3: Main Data Entities and Relationships

The database schema comprises relational entities designed for high-frequency time-series telemetry, granular cell matrices, and alert tracking:

Entity (Table)	Purpose	Important Fields & Relations
BatteryPack (thermocore_battery_packs)	Stores physical battery pack master records, chemistry specifications, and thermal operating boundaries.	id (PK), packCode (Unique), packName, vehicleModel, chemistry (LFP/NMC/SolidState), capacityKWh, nominalVoltage, cellCount, coolingType, maxSafeTemp, criticalTemp, status. Relation: 1:N with SensorReading, 1:N with ThermalAlert.
SensorReading (thermocore_sensor_readings)	Captures periodic time-series sensor observations of battery and micro-channel coolant dynamics.	id (PK), batteryPackId (FK), timestamp, batteryTemp, minCellTemp, maxCellTemp, tempGradient, voltage, current, coolantInletTemp, coolantOutletTemp, coolantFlowRate, stateOfCharge, stateOfHealth, thermalRiskLevel, isHotspotDetected, testScenario. Relation: 1:N with CellTemperature.
CellTemperature (thermocore_cell_temperatures)	Stores individual cell temperature readings across 96–144 cell grid arrays for hotspot localization.	id (PK), sensorReadingId (FK), cellIndex (0..N-1), cellLabel (e.g., C-01), temperature (°C), isHotspot (Boolean), deltaFromAvg (°C). Relation: N:1 with SensorReading.
ThermalAlert (thermocore_thermal_alerts)	Maintains active and historical thermal anomaly alerts with technician lifecycle tracking.	id (PK), batteryPackId (FK), sensorReadingId (FK), alertType, severity (INFO, WARNING, DANGER, CRITICAL), description, detectedParameter, observedValue, recommendedAction, status (ACTIVE, ACKNOWLEDGED, RESOLVED), timestamp. Relation: N:1 with BatteryPack.

Task 4: Design the REST API (Primary Resource: BatteryPack & Telemetry)

Method	Endpoint	Purpose	Request / Response Format

Method	Endpoint	Purpose	Request / Response Format
GET	`/api/batteries`	List all registered battery packs with thermal status.	None -> JSON array of BatteryPack objects [200 OK]
GET	`/api/batteries/{id}`	Retrieve single pack specifications and metadata.	Path variable id -> Single BatteryPack JSON object [200 OK / 404]
POST	`/api/batteries`	Register a new EV battery pack in database.	JSON payload with pack details -> Saved BatteryPack entity with generated ID [201 Created]
PUT	`/api/batteries/{id}`	Update battery pack specifications or thresholds.	Path variable id + JSON payload -> Updated BatteryPack object [200 OK]
DELETE	`/api/batteries/{id}`	Delete a battery pack and cascade readings.	Path variable id -> Success confirmation message [200 OK]
POST	`/api/sensors/readings`	Ingest real-time multi-channel sensor telemetry.	SensorInputDto payload -> Processed SensorReading with risk score and alerts [201 Created]

Task 5: GitHub Repository and Project Structure

- **Repository Link:** https://github.com/Shravani-3005/Java-Ev-battery-project (Branch: main)
- **Latest Commit Hashes:** bfc0436 (Supabase cloud pooler update), 28e52ae (Deployment documentation), c8844b6 (Complete full-stack foundation commit).

Project Directory Organization:

```
Java-Ev-battery-project/
├── backend/                    # Java 21 / Spring Boot 3 Microservice
│   ├── src/main/java/com/thermocore/
│   │   ├── config/             # CorsConfig, DataInitializer
│   │   ├── controller/         # Battery, Sensor, Thermal, Alert, Sim Controllers
│   │   ├── dto/                # SensorInputDto, ThermalAnalysisResultDto, etc.
│   │   ├── entity/             # BatteryPack, SensorReading, CellTemp, ThermalAlert
│   │   ├── repository/         # Spring Data JPA Repositories
│   │   ├── service/            # Battery, Alert, Analytics, Recommendation Services
│   │   ├── simulation/         # SensorSimulationService (Multi-scenario generator)
│   │   └── thermal/            # ThermalIntelligenceService (Rule-based engine)
│   ├── src/main/resources/     # application.properties, application-postgres.properties
│   ├── Dockerfile              # Multi-stage Temurin-21 container build for Render
│   └── pom.xml                 # Maven configuration and dependencies
├── frontend/                   # React 18 / Vite Single-Page Application
│   ├── src/                    # App.jsx, main.jsx, index.css (Industrial dark palette)
│   ├── src/api/client.js       # Axios client with dynamic VITE_API_URL fallback
│   ├── src/components/         # Navbar, Sidebar, KPICard
│   ├── src/pages/              # 9 Full Pages (Dashboard, Batteries, Live, Sim, etc.)
│   ├── tailwind.config.js      # Custom theme colors (#0B0B0D, #FF7900, #C0C0C0)
│   └── vercel.json             # Vercel SPA HTML5 rewrite rules
├── database/schema.sql         # PostgreSQL DDL script for Supabase
├── docs/                       # ARCHITECTURE.md, DEPLOYMENT_GUIDE.md
├── render.yaml                 # Render Cloud Blueprint for zero-downtime deployment
└── README.md                   # Complete architectural guide and setup instructions
```

Task 6: AI / Intelligent Feature Specification (Design Only for Phase 1)

The thermal management platform specifies an intelligent classification and advisory subsystem. In Phase 1, this logic is implemented as a deterministic rule-based heuristic service in Spring Boot, providing an extensible integration interface for a trained machine learning model in Phase 2.

- **1. Input(s) Used by Intelligent Feature:** Pack average temperature (°C), maximum cell temperature (Tmax), minimum cell temperature (Tmin), inter-cell temperature gradient (ΔT = Tmax - Tmin), pack current (C-rate load), coolant inlet temperature (Tin), coolant outlet temperature (Tout), coolant flow rate (LPM), and State of Charge (SOC).

- **2. Output / Recommendation / Classification Produced:** Thermal Risk Level (NORMAL, MODERATE, HIGH RISK, CRITICAL), Hotspot Flag with anomalous cell indices, and Recommended Cooling Actions (Target pump flow rate in LPM, radiator fan PWM percentage, and powertrain power derating percentage).
- **3. Who Uses the Output and What Decision it Supports:** Used by the vehicle Electronic Control Unit (ECU) / Battery Management System (BMS) for autonomous pump valve actuation, and by fleet maintenance supervisors to schedule immediate coolant inspections or cell balance servicing.
- **4. How Spring Boot Obtains the Result:** Phase 1 implements ThermalIntelligenceService.java in Spring Boot utilizing thermodynamic boundary heuristics. Phase 2 will seamlessly integrate an external Python FastAPI microservice hosting a Scikit-Learn Random Forest / XGBoost classifier via a non-blocking REST client.
- **5. Fallback and Uncertainty Strategy:** Fail-safe architecture: if the external AI service is unreachable, times out (>500ms), or returns prediction confidence below 80%, the Spring Boot backend automatically degrades to hardcoded thermodynamic safety boundaries. If telemetry sensors disconnect entirely, a CRITICAL sensor-loss alert is logged and coolant pumps default to 100% maximum fail-safe flow.
- **Important Scope Transparency:** Phase 1 strictly utilizes validated thermodynamic heuristics and algebraic rules in Java. These are not falsely labeled as artificial intelligence or trained machine learning models.

Steps for execution:

1. Continue with the case study selected earlier; do not spend the session selecting a new topic.
2. Freeze the MVP, users and 3-5 core features; select one primary workflow for today.
3. Draw the architecture and identify the main data entities and API endpoints.
4. Create/organize the project repository with separate frontend and backend folders/projects.
5. Implement the backend model/repository/controller and connect it to the database.
6. Implement a React input/display component and call the backend using Axios or Fetch.
7. Demonstrate that data is stored/retrieved persistently and displayed in React.
8. Commit and push the working code and record the required evidence in this writeup.

Conclusion:

Phase 1 of the THERMOCORE EV Battery Thermal Intelligence & Monitoring System successfully established a robust, end-to-end full-stack foundation. By scoping the institute case study (KJS-CES-02) into an actionable MVP, the team successfully developed and demonstrated a working vertical slice spanning a responsive React 18 frontend, a modular Java 21 Spring Boot REST backend, and persistent Supabase Cloud PostgreSQL database storage.

The implementation verified critical full-stack capabilities including asynchronous REST communication with Axios, declarative component state management, transactional JPA entity mapping, and automated database table generation. The thermal intelligence architecture was cleanly delineated: Phase 1 provides deterministic rule-based analysis and realistic sensor simulation, establishing verified API contracts and data schemas that will seamlessly accommodate trained machine-learning models in subsequent project phases.

Postlab questions:

1. What is the MVP of your selected case study, and which user problem does it solve?

Answer: The Minimum Viable Product (MVP) of THERMOCORE is a centralized web platform that continuously ingests multi-cell battery telemetry and micro-channel liquid coolant parameters, evaluates thermal runaway risk using rule-based thermodynamic boundaries, and displays real-time battery status, hotspot anomalies, and cooling recommendations on an industrial React dashboard. It solves the critical automotive problem of unmonitored localized heat accumulation in EV battery packs. In traditional EVs, aggregate pack temperature sensors often conceal localized cell overheating, which can propagate into catastrophic thermal runaway. THERMOCORE provides granular cell monitoring, proactive risk classification, and actionable cooling pump recommendations for battery engineers and fleet operators.

2. Why did your team choose the implemented feature as the first end-to-end vertical slice?

Answer: The team selected the Battery Pack Management and Telemetry Ingestion workflow as the initial vertical slice because it establishes the foundational data pipeline upon which all subsequent analytical and intelligence features depend. In battery thermal engineering, neither thermal risk evaluation, hotspot detection, nor automated cooling advisories can function without valid battery pack specifications (chemistry, cell count, thermal limits) and active sensor readings. Delivering this vertical slice required implementing every tier of the modern full-stack architecture—React form controls and state hooks, Axios HTTP communication, Spring Boot @RestController endpoints, transactional service validation, and PostgreSQL relational table persistence with foreign keys—proving system feasibility early.

3. Explain how data travels from the React interface to Spring Boot and the database in your implementation.

Answer: Data traversal follows a structured, asynchronous client-server pipeline:

- **a. UI Event & State Capture:** A user enters battery parameters (e.g., pack code, vehicle model, capacity, chemistry) in React form components managed by useState hooks.
- **b. HTTP Client Dispatch:** Upon form submission, an Axios client (configured in client.js with dynamic VITE_API_URL routing) serializes the state into a JSON payload and dispatches an asynchronous HTTP POST request to /api/batteries.
- **c. Controller Ingestion:** The request reaches Spring Boot's DispatcherServlet, which routes it to BatteryController.java. The controller deserializes the JSON into a validated BatteryPack entity using Jackson and invokes BatteryService.createBatteryPack().
- **d. Service & JPA Persistence:** The service layer enforces business logic and calls BatteryPackRepository.save(). Spring Data JPA and Hibernate generate the SQL INSERT statement and execute it through the HikariCP connection pool over an SSL socket to Supabase Cloud PostgreSQL.
- **e. DB Commit & UI State Update:** PostgreSQL inserts the record into thermocore_battery_packs, generates a sequence primary key, and commits the transaction. The entity is returned to React as a 201 Created JSON response and prepended to the active state table without page refresh.

4. Which entities/resources are central to the project, and why?

Answer: The central entities are BatteryPack, SensorReading, CellTemperature, and ThermalAlert:

- **1. BatteryPack (thermocore_battery_packs):** Represents the physical battery hardware (e.g., LFP, NMC, Solid-State) and anchors all thermal boundaries, maximum safe temperatures, and cell counts. All telemetry observations and alerts maintain foreign key dependencies to this entity.
- **2. SensorReading (thermocore_sensor_readings):** The core time-series entity recording high-frequency operational measurements including pack temperature, voltage, current, coolant inlet/outlet temperatures, flow rate (LPM), SOC, and calculated heat generation (Watts). It captures operational state over time.
- **3. CellTemperature (thermocore_cell_temperatures):** Provides granular spatial resolution across the 96–144 cell matrix. By associating individual cell temperatures with a sensor reading, the system can compute inter-cell thermal gradients (ΔT) and identify isolated hotspots that pack-level averages miss.
- **4. ThermalAlert (thermocore_thermal_alerts):** Tracks thermal exceptions, overheating events, and sensor failures. It records severity levels (INFO, WARNING, DANGER, CRITICAL), technician acknowledgments, and resolution timestamps, closing the operational loop.

5. What AI/intelligent functionality is planned, what inputs will it use, and what output will it produce?

Answer: The planned intelligent module is a predictive thermal runaway classifier and dynamic cooling optimization model. In Phase 2, this will be implemented as an external Python microservice running a Scikit-Learn Random Forest or XGBoost algorithm trained on EV drive-cycle battery datasets. It will take as inputs: multi-cell temperatures, rate of temperature rise (dT/dt), inter-cell gradient (ΔT), battery C-rate, coolant inlet/outlet temperature differential, and ambient conditions. It will output: (1) a predictive Thermal Risk Classification 60 seconds before critical thresholds are reached, (2) an early hotspot probability score for individual cells, and (3) an optimized coolant pump flow rate and radiator fan duty cycle that minimizes parasitic pump energy consumption while preventing thermal runaway.

6. Which parts of the final project remain to be implemented after Phase 1?

Answer: While Phase 1 successfully delivered the full-stack foundation, the following modules are planned for subsequent phases:

- **1. Machine Learning Model Integration:** Replacing the current Java-based rule engine with an external Python FastAPI microservice running trained machine learning classification models.
- **2. Real-Time WebSocket Streaming:** Upgrading the current periodic HTTP polling mechanism to a full-duplex WebSocket (STOMP over SockJS) connection for sub-second telemetry streaming.
- **3. Security & Authentication:** Implementing Spring Security with JSON Web Tokens (JWT) and role-based access control (RBAC) to differentiate between Engineers, Technicians, and Admin users.
- **4. Advanced Simulation Profiles:** Expanding the simulation suite to include cold-weather preheating, DC ultra-fast charging thermal spikes, and micro-channel physical blockage simulations.
- **5. Fleet Export & Reporting:** Automated PDF export of historical battery health reports and compliance logs for automotive regulatory audits.

Outputs to be Attached / Recorded (Evidence Artifacts):

The following labelled placeholders record evidence of the working full-stack implementation, verifying frontend UI rendering, backend REST controller responses, database persistence in Supabase PostgreSQL, and GitHub version control:

[EVIDENCE SCREENSHOT PLACEHOLDER: REACT INDUSTRIAL DASHBOARD INTERFACE]

Figure 1: React Industrial Dashboard Interface

Operational Scope: Demonstrates the full-stack dashboard featuring live KPI summary cards (Total Packs, Normal Batteries, Critical Alerts, Avg Temperature), temperature trend lines, risk distribution donut chart, and active EV pack fleet overview.

`Verification Endpoint / Source: frontend/src/pages/DashboardPage.jsx | http://localhost:5174/ (Vite Development Server)`

[EVIDENCE SCREENSHOT PLACEHOLDER: BATTERY PACK MANAGEMENT & REGISTRATION INTERFACE]

Figure 2: Battery Pack Management & Registration Interface

Operational Scope: Demonstrates the battery pack management interface displaying registered packs (Tata Ultra T.7 Bus, Orion Racing Prototype, Mahindra Zor Van, AERIS Hyper-GT) with chemistry, capacity, cell count, and cooling type.

`Verification Endpoint / Source: frontend/src/pages/BatteryPacksPage.jsx | End-to-End Vertical Slice Entrypoint`

[EVIDENCE SCREENSHOT PLACEHOLDER: SUCCESSFUL REST API REQUEST & RESPONSE (SWAGGER / POSTMAN)]

Figure 3: Successful REST API Request & Response (Swagger / Postman)

Operational Scope: Demonstrates successful HTTP GET request to /api/batteries and /api/dashboard/summary returning 200 OK with formatted JSON payload containing live battery pack parameters and aggregated thermal KPIs.

`Verification Endpoint / Source: backend/src/main/java/com/thermocore/controller/BatteryController.java | http://localhost:8082/swagger-ui.html`

[EVIDENCE SCREENSHOT PLACEHOLDER: RETRIEVED DATABASE RECORD DISPLAYED IN REACT UI]

Figure 4: Retrieved Database Record Displayed in React UI

Operational Scope: Demonstrates live client-side data rendering following an asynchronous Axios GET request, proving that battery pack records persisted in the backend database are successfully retrieved and displayed.

`Verification Endpoint / Source: frontend/src/api/client.js (axios instance) -> Spring Boot DispatcherServlet -> React State`

[EVIDENCE SCREENSHOT PLACEHOLDER: POSTGRESQL PERSISTENT DATABASE RECORDS IN SUPABASE CLOUD]

Figure 5: PostgreSQL Persistent Database Records in Supabase Cloud

Operational Scope: Demonstrates persistent records stored in the cloud PostgreSQL database (thermocore_battery_packs, thermocore_sensor_readings, thermocore_cell_temperatures), verifying relational integrity and schema creation.

`Verification Endpoint / Source: Supabase Project: aws-0-eu-west-1.pooler.supabase.com:5432/postgres | database/schema.sql`

[EVIDENCE SCREENSHOT PLACEHOLDER: GITHUB REPOSITORY STRUCTURE AND COMMIT HISTORY]

Figure 6: GitHub Repository Structure and Commit History

Operational Scope: Demonstrates the public repository structure with separate /frontend, /backend, /database, and /docs directories, showing verified commits establishing the full-stack foundation.

`Verification Endpoint / Source: https://github.com/Shravani-3005/Java-Ev-battery-project | Branch: main | Commit: bfc0436`

Technical Implementation Status & Evidence Matrix:

The following matrix provides an authoritative, evidence-backed classification of project features, distinguishing fully implemented functionality from future project phases:

Functional Subsystem	Implementation Status	Verification Evidence & Source Reference
Battery Pack CRUD Management	**IMPLEMENTED (Fully Operational)**	BatteryController.java, BatteryService.java, BatteryPackRepository.java, BatteryPacksPage.jsx. Verified via REST endpoints (/api/batteries) with persistent Supabase PostgreSQL storage.
Sensor Telemetry & Simulation	**IMPLEMENTED (Fully Operational)**	SensorSimulationService.java generates realistic battery temp, voltage, current, and coolant dynamics across 5 scenarios. Verified via /api/simulation/* and LiveMonitoringPage.jsx.
Cell Hotspot & Gradient Analysis	**IMPLEMENTED (Rule-Based Heuristic)**	ThermalIntelligenceService.java analyzes 96–144 cell arrays, calculating ΔT = Tmax - Tmin and flagging hotspots (ΔT > 5°C). Verified via HotspotDetectionPage.jsx.
Cloud PostgreSQL Persistence	**IMPLEMENTED (Verified in Production)**	Supabase Cloud PostgreSQL database connected via HikariCP pool with SSL mode require. Schema auto-generated by Hibernate DDL (application-postgres.properties).
Machine Learning Model	*PLANNED (Phase 2 Design)*	Designed in Task 6. Phase 1 intentionally uses validated Java thermodynamic heuristics. External Python FastAPI Random Forest microservice scheduled for Phase 2.